Receipt Capture Web SDK — Technical Specification

`ReceiptCaptureSDK` is a dependency-free, browser-based SDK that captures a receipt with the device camera (or an uploaded image), prepares an OCR-optimized image, and runs it through a cloud backend powered by **Amazon Textract** — returning structured receipt data (vendor, date, total, line items).

> **How to read this document**
>
> | If you are… | Read |
> | --- | --- |
> | A **stakeholder** / **lead** wanting to know what it does | **Part 1 — Overview** |
> | A **developer integrating** the SDK into an app | Parts **1 + 2** |
> | A **developer maintaining** / **extending** the SDK | Parts **1 + 3 + 4** |

Part 1 — Overview

Audience: everyone.

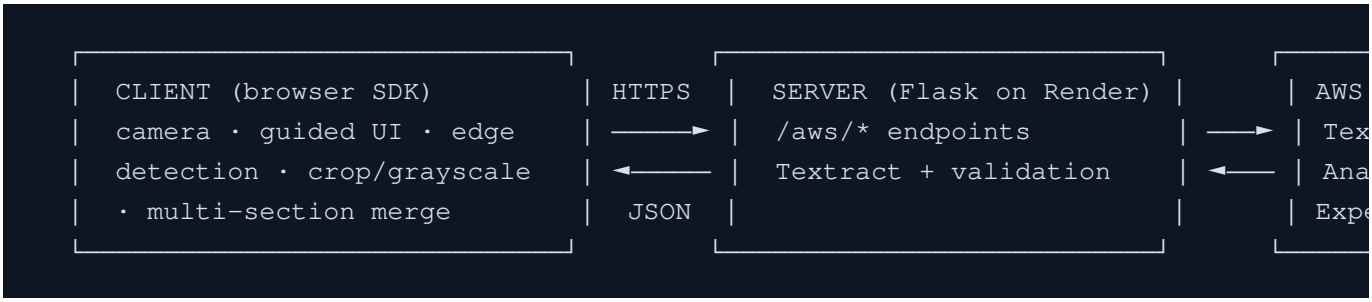
What it is

A single JavaScript file (`static/receipt-capture-sdk.js`) that you drop into a web page. It opens a full-screen, guided camera experience, helps the user frame the receipt, captures an optimized image, and sends it to a backend that extracts the receipt's contents with Amazon Textract.

What it does

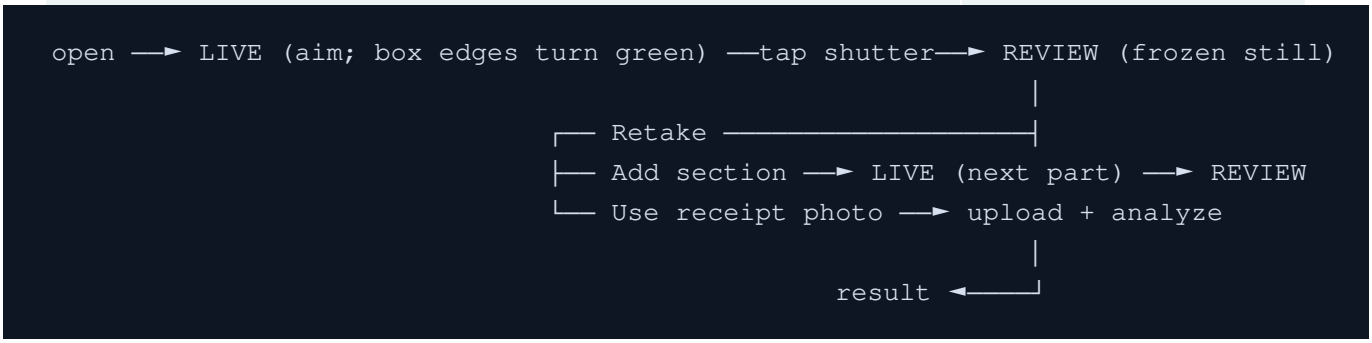
- **Guided camera capture** — rear camera, a fixed alignment box whose edges turn green when the receipt is framed correctly, live lighting coaching, and a flash toggle. Capture is manual (the user taps the shutter).
- **On-device quality checks** — brightness/glare checks and edge detection so only a real, readable receipt is captured.
- **Long receipts** — captured in multiple sections; the results are merged into one clean item list (duplicates from the overlap removed, real repeat purchases kept).
- **Image preparation** — crops to the receipt, corrects rotation, converts to a compressed grayscale JPEG sized for OCR.
- **Secure delivery** — uploads to the backend and polls for the result; the browser never holds cloud credentials.

Architecture at a glance (cloud-hybrid)



Security boundary: all OCR, validation, and (when enabled) fraud/dedup logic runs server-side. The client only prepares and delivers the image. The browser talks only to the app's own `/aws/*` endpoints — never to AWS directly.

The capture experience



Part 2 — Integration Guide

Audience: developers adding the SDK to an app.

Requirements

- **Secure context** — the camera requires `https://` (or `http://localhost`).
- A **modern browser** with `getUserMedia` (iOS Safari, Android Chrome, desktop Chrome/Edge/Firefox/Safari).
- A reachable **backend** exposing the `/aws/*` endpoints (see Part 3) with CORS allowing your page's origin.

Quick start

```
<!-- 1. Point at your backend, then load the SDK -->
<script>window.BACKEND_URL = "https://your-backend.example.com";</script>
<script src="receipt-capture-sdk.js"></script>
```

```
// 2. Offer the camera only on touch devices; desktop should upload instead.
const canUseCamera = window.matchMedia("(pointer: coarse)").matches
  && !!(navigator.mediaDevices && navigator.mediaDevices.getUserMedia);

// 3. Launch the guided capture and use the result.
```

```
async function scanReceipt() {
  const base = (window.BACKEND_URL || "").replace(/\/$/, "");
  const sdk = new ReceiptCaptureSDK({
    userId: "member-123",
    endpoints: {
      uploadUrl: base + "/aws/get-upload-url",
      status:    base + "/aws/analyze-status",
    },
  });

  try {
    const receipt = await sdk.captureFlow();   // opens the full-screen UI
    console.log(receipt.vendor.name, receipt.total);
    receipt.line_items.forEach(li => console.log(li.description, li.total));
    // → credit cashback, match offers, etc.
  } catch (err) {
    if (err.cancelled) return;   // user closed the camera
    alert(err.message);          // camera couldn't start
  }
}
```

The result object (what `captureFlow()` resolves to)

```
{
  "vendor": { "name": "WHOLE FOODS MARKET" },
  "date": "06/13/2026",
  "total": 32.54,
  "currency_code": "USD",
  "document_type": "receipt",
  "line_items": [
    { "description": "OG HONEYCRISP APPLE", "total": 2.39, "quantity": "0.35 lb" }
  ],
  "line_item_count": 11,
  "avg_confidence": 95.38,
  "source": "aws-textract",
  "merged_sections": 2
}
```

Error handling

`captureFlow()` resolves with the result on success. Otherwise:

Outcome	How it surfaces
User closes the camera	Promise **rejects** with `{ cancelled: true }`
Camera can't start (no permission, not HTTPS)	Promise **rejects** with an `Error`

Server rejects the receipt (unreadable, low confidence, too old)	Shown **inside the SDK's UI** (error panel with Retake / Close), the user retakes or closes (→ `cancelled`)

So your `try/catch` only needs to handle **cancel** and **camera-start failure** — analysis rejections are presented to the user in the capture UI itself.

Options

Option	Default	Description
`endpoints.uploadUrl`	`/aws/get-upload-url`	Endpoint returning a job id + PUT URL
`endpoints.status`	`/aws/analyze-status`	Poll endpoint
`userId`	`"demo-user"`	Sent as `X-User-Id` on upload (used by dedup)
`idealWidth` / `idealHeight`	`1920` / `1080`	Requested capture resolution
`jpegQuality`	`0.85`	Output JPEG quality
`grayscale`	`true`	Convert output to grayscale
`lumaMin` / `lumaMax`	`40` / `230`	Luminance guardrail bounds
`glareMax`	`0.14`	Near-white fraction before a glare warning
`edgeDetection`	`true`	Enable OpenCV edge detection
`box`	`{left:0.08, right:0.92, top:0.11}`	Fixed alignment box (viewport fractions)
`alignTol`	`0.12`	±tolerance for an edge to count as aligned (green)
`contrastMin` / `textureMin`	`26` / `0.035`	Real-receipt detection gates
`pollIntervalMs` / `maxPolls`	`2000` / `60`	Status polling cadence / ceiling (~2 min)
`opencvUrl`	`…/opencv.js`	OpenCV.js source (lazy-loaded)

Methods & events

Method	Description
`new ReceiptCaptureSDK(options)`	Create an instance
`captureFlow()` → `Promise<result>`	Open the guided UI; resolve with the OCR result

`on(event, cb)`	Subscribe to progress events
`close()` / `stop()`	Tear down the UI / stop the camera

Event	Payload	When
`ready`	`{ width }`	Camera started
`segment`	`{ count, luminance }`	A section was captured
`status`	`{ phase, message, attempt?, status? }`	Pipeline progress (`uploading`, `polling`, `detector-ready`, …)

Device gating (desktop vs mobile)

The camera should be offered **only on touch devices**; desktops should upload an image instead (sent through the same backend). Detect with `matchMedia("(pointer: coarse)")` (this treats iPads as tablets and mouse-driven touch laptops as desktops). The upload path uses the same `get-upload-url → PUT → poll` endpoints with the chosen file as the body.

Part 3 — Technical Internals

Audience: developers maintaining or extending the SDK.

3.1 Camera constraints

Requests the rear camera with `facingMode:{exact:"environment"}` and `advanced:[{focusMode:"continuous"}]`; falls back to `facingMode:"environment"` then the default sensor. Ideal resolution 1920×1080. The `<video>` is forced `playsinline`/`autoplay`/`muted` (attributes **and** properties) for iOS Safari.

3.2 Edge detection (OpenCV.js)

- **Lazy-loaded** (~8 MB WASM) from a CDN when the camera opens; falls back to the fixed guide box (no green/▼ feedback) if it fails to load.
- **Brightness-band** detection — the receipt is lighter than its background — not gradient-based, which would mistake interior text for an edge.
- **False-positive gates:** (1) a contrast gate (the bright band must exceed the background) and (2) a Canny **text-density** gate (smooth surfaces like walls or hands are rejected).
- **Per-edge state:** an edge is "in view" only when the band starts/ends inside the frame; it turns **green** when that edge lands within `alignTol` of the fixed box edge. The **bottom** line appears only when the receipt's end is detected.

3.3 Image processing

- **DPR-aware crop:** the fixed box is mapped onto the raw sensor matrix via `getBoundingClientRect()` ratios and rasterized at `devicePixelRatio`.

- **Skew correction:** landscape sensor + portrait layout → rotate the canvas 90° CW.
- **Luminance guardrail:** mean Rec.709 luminance must be within `[lumaMin, lumaMax]`.
- **Output:** grayscale JPEG at `jpegQuality`.

3.4 Multi-section merge

Sections are **not pixel-stitched** (fragile on thermal paper). Each section is sent to Textract independently, then results are merged:

- The capture overlap is a contiguous run of lines shared between sections.
- The merge finds where the next section's leading run matches a contiguous run **anywhere** in the accumulated items (matched by description, since OCR totals drift) and drops that run **once**.
- Because it matches *runs*, legitimately repeated purchases are preserved.

3.5 Backend API (`/aws/*`)

Flask + boto3 (app.py).

Endpoint	Purpose
`GET /aws/status`	Whether boto3 + AWS credentials are available
`POST /aws/get-upload-url`	Returns `{ job_id, key, upload_url }` (absolute, CORS-friendly)
`PUT /aws/upload/<job_id>`	Receives image bytes; starts Textract in a background thread
`GET /aws/analyze-status?job=<id>`	`{ status: processing|done|rejected|failed, result?, error? }`

Server-side validation layer: readability hard block · confidence gate (`TEXTRACT_MIN_CONFIDENCE`, default 50) · temporal rules (future-dated / older than `RECEIPT_MAX_AGE_DAYS`, default 14) · line-item cleanup (strips Textract phantom rows + summary/payment lines; does **not** dedup by description, so repeats survive) · deduplication (SHA-256 image hash + `user_vendor_total_date` semantic key) — **currently commented out / gated by `DEDUP_ENABLED`** during prototyping.

3.6 Code map

File	Contains
`static/receipt-capture-sdk.js`	The SDK: UI build, camera, `_detectEdges`, `_captureBox`, `submit`/`_analyzeBlob`/`_mergeResults`
`app.py`	`/aws/*` routes, `normalize_textract`, `run_analyze` (validation), `_clean_line_items`
`design_preview_noclip.html`	Reference integration: device gating, tray, upload path, result rendering

Part 4 — Limitations & Production Roadmap

Known limitations

- **Edge/bottom detection** needs the receipt lighter than its background — best on a dark, non-shiny surface; weak on white tables. A heavier on-device ML model would improve robustness.
- **Overlap merge matches by description** — if the same line is OCR'd differently between sections, a duplicate can slip through. Fuzzy (edit-distance) matching would absorb that drift.
- **OpenCV.js is ~8 MB**, lazy-loaded on first camera open; on slow networks edge detection activates a few seconds in (capture still works meanwhile).
- **Textract synchronous** `AnalyzeExpense` supports JPEG/PNG, not PDF.

Production extension points

The current backend is a lightweight stand-in: Textract runs synchronously on the uploaded bytes (no S3), and jobs/dedup live in process memory. A production build would use **API Gateway + Lambda + an S3 presigned PUT + asynchronous Textract + DynamoDB** for the dedup indexes. The client pipeline (presigned PUT + polling) already matches that shape, so **only the server changes** — the SDK is unaffected.

Configuration (backend env)

`AWS_REGION` · AWS credentials (env or IAM role) · `ALLOWED_ORIGIN` (CORS) · `RECEIPT_MAX_AGE_DAYS` · `TEXTRACT_MIN_CONFIDENCE` · `DEDUP_ENABLED`.

Tech stack

Vanilla ES + Canvas 2D + `getUserMedia` (SDK) · OpenCV.js 4.8 (WASM, lazy) · Flask + flask-cors + boto3 + gunicorn (backend) · Amazon Textract `AnalyzeExpense` · GitHub Pages (frontend) + Render (backend).